Autonomous Bug-Squashing Agent

A Planner / Executor / Critic Loop with a Multi-Model, OSS-Grounded Benchmark Harness

CMPE 258 — Deep Learning, Spring 2026 • San José State University

Project Track: Application / *Focused Areas: LLM agent systems · Automated program repair · Multi-model benchmarking*

Name	SJSU ID	Email
Pranav Jitendra Trivedi	019089512	pranavjitendra.trivedi@sjsu.edu
Yashashav Devalapalli Kamalraj	017856371	yashashav.devalapallikamalraj@sjsu.edu
Saransh Soni	019115122	saransh.soni@sjsu.edu

Per-module contribution breakdown in § 5.

Abstract

In this paper we propose a Python bug-fixing agent that takes a Python file and its PyTest file and edits the source code till it passes. The agent runs a Planner / Executor / Critic loop. The Planner is a LLM that picks one of the three tools (`read_file, edit_file, and run_bash`). The Executor writes the correction using one of three tools (temp file, `ast.parse, os.replace`) in the constraints of the case. The Critic returns on of three judgements (`RESOLVED, RETRY, UNRESOLVED`) and then feeds the diff and summary of the failure, if any, to the next turn. A memory module has also been made that prevents future turns from exploring solutions already unsuccessful by earlier attempts.

Two evaluation benchmarks have been used to evaluate the agent. A hand-curated 52 Python folders, split between syntax, logic and scope type errors, and a harder benchmark, where bugs were injected into `pallets/click v8.3.2`. Gemini 3 Flash and Pro, Qwen 2.5 72B and Gemma 3/4 models were used in the benchmark. Best result on the curated dataset, at 53.8 % success rate, was with Qwen's model; Gemma 4 reached 100 % on every small (≤ 8 case) named run and 75–83 % on the larger `deep12` sweeps. Gemini 3 Pro solved bug with 33 % accuracy on the OSS benchmark, while all other models failed to have any success. Our findings indicate that the underlying model architecture determines the larger success rate, and the contribution of this work is in increasing task-completion efficiency.

1 Introduction & Problem Description

In the contemporary context of AI-assisted software development, inputting a snippet of faulty code in a chat assistant may worsen it. Software such as Devin, SWE-Agent, and Aider are available; however, their processes are hard to understand, and engineers tend not to trust whether their corrections have been successful or not.

The aim of this project is to structure the entire process into a structured agent system that will a) detect the location of the bug, b) provide a patch to correct the code, c) confirm the patch's validity through regression tests, and describe the failed attempts in its processes. It is modular and can be used with both local inference and cloud API-based systems.

Evaluation of Automated Program Repair (APR) tools are largely bound within their evaluation. Therefore, transparency is essential. Reporting framework for this project involves illustrating how the agent works when faced with random and unseen inputs. Target audience includes CS students learning debugging, educators who need an auditable tutor, and researchers interested in APR baselines.

2 Background & Related Work

Work	**Relation to ours**
SWE-Bench (Jimenez et al., 2024)	Real GitHub PRs as oracle. Heavy infra; we kept the spirit at a single-repo scale we could actually run, via `pallets/click`.
SWE-Agent (Yang et al., 2024)	ACI (agent-computer interface) with shell tools. Influenced our `read_file`/`edit_file`/`run_bash` triplet.
Aider, Devin, OpenHands	Production agents with closed loops and opaque metrics. Ours logs every step as JSON lines and accounts tokens per call, so the loop is inspectable from outside.
CodeR, AgentCoder	Multi-role debate frameworks. We chose Planner / Executor / Critic to keep the role count tractable.
Mutation testing (Just et al.)	Foundation of `SyntheticMutationAdapter`: AST-level operator/literal/name flips against a pinned commit.
Wilson score interval (Wilson, 1927)	Used in `benchmark/analyze.py` for small-sample 95 % confidence intervals on pass rates.
Ollama / Gemma 4	Local inference. Runs on a laptop with no API key, which matters when the code can't leave the machine.

LangChain, AutoGen, and CrewAI not in dependencies. The code for the agent is all hand-written Python. This was a requirement in the proposal since the goal was to make sure that any bugs in the loop could be traced back easily.

3 System Design

3.1 The Planner / Executor / Critic Loop

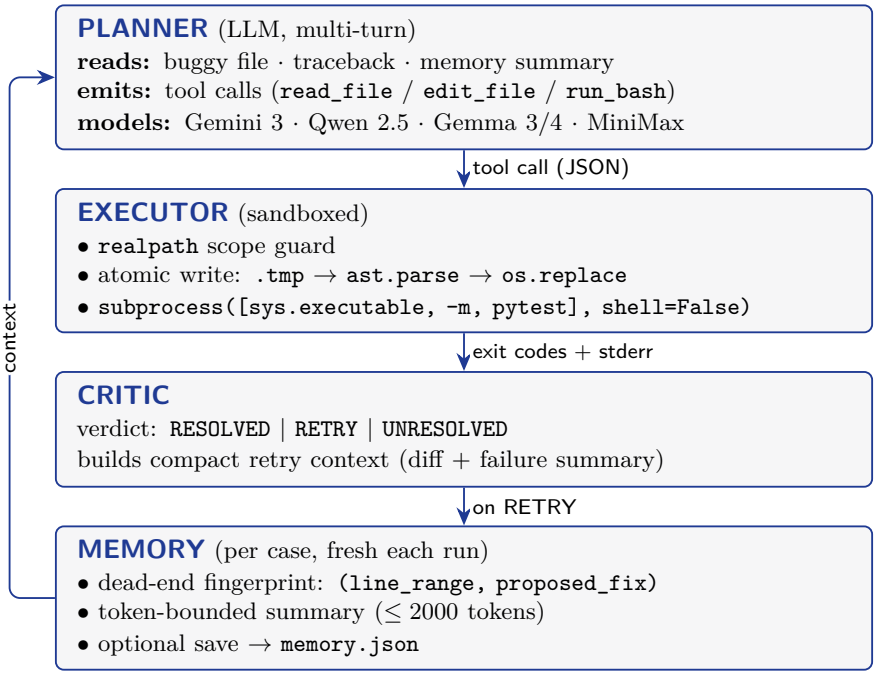

Figure 1: Planner / Executor / Critic loop with per-case Memory. Fresh `Memory()` per case run; cross-case state leakage is treated as a bug.

Design choices. Every invocation of `run_case_with_pec()` will create an instance of `Memory()`; all others are stateless and any stateful corruption between cases would be a bug. Patches are actual line edits, not diffs; proposed_fix replaces lines [`lo`,`hi`], both inclusive, and any invalid python code will not pass `ast.parse` but be rejected prior to `os.replace` being invoked. The agent is given three methods to use: `read_file`, `edit_file`, and `run_bash`, with the restriction that only `pytest` calls can be made through `run_bash()`. The dead-end fingerprint is `(line_range_tuple, proposed_fix.strip())`.

3.2 Benchmark Pipeline

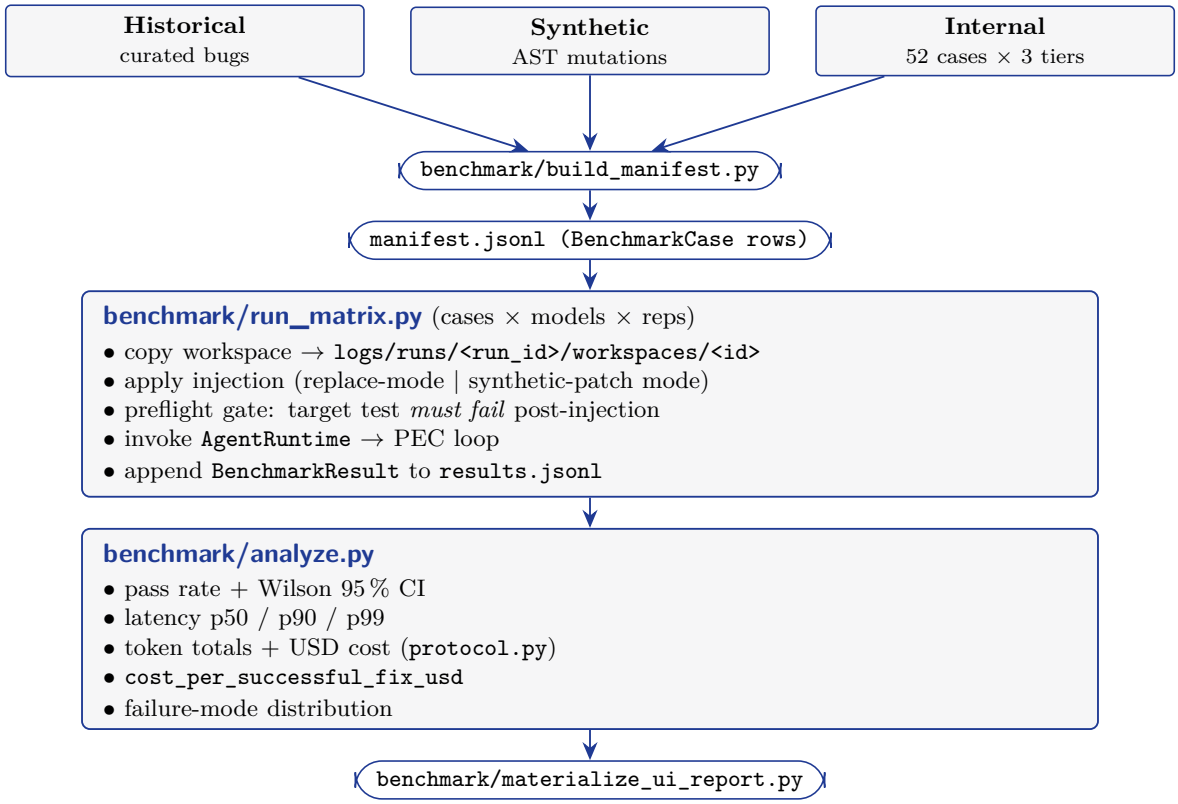

Figure 2: Benchmark pipeline: hybrid manifest construction → run matrix → analysis → UI report.

Two adapters. These two adapters target failure modes that are different. With synthetic mutations, we can measure capability in terms of a controlled distribution. The OSS adapter places the agent in an actual codebase, complete with multi-module import, large files, and fixture setup, where most of the surprises come from code we didn't write.

3.3 Models

Alias	Backend	Typical role
`gemini` (3 Pro / 3 Flash)	`google-genai` SDK	Cloud reasoning baseline
`qwen` (Qwen 2.5 72B Instruct)	OpenRouter REST	Open SOTA cloud
`minimax` (M-2.5)	Together AI REST	Open MoE comparison
`gemma4`, `gemma3`, `gemma3:4b`	Ollama (`127.0.0.1:11434`)	Local / private

All implement the `models/base.py::BaseModel` ABC: `complete(prompt)` → `ModelResponse(text, input_tokens, output_tokens, latency_ms)`.

4 Implementation Details

4.1 Stack

Python 3.11 was used as it contains the click library `click 8.x`, which is vendored in `external/`, and this project utilizes several core libraries including `google-genai`, `requests`, `fastapi`, `uvicorn`, `pytest`, `rich`, and `python-dotenv`. Each inference route uses either Ollama (for local runs with Gemma 3/4), OpenRouter (for Qwen), Together AI (for MiniMax) or Google AI Studio (for Gemini). Only Ollama does not require an API Key. At the end of each run, the case workspace is copied into `logs/runs/<run_id>/workspaces/<attempt_id>` so that the `dataset/cases/case_NNN/buggy.py` never changes due to mutation. Structured logging is done using JSON lines by `logger.py`. Both `latest_results_path.txt` and `latest_run_dir_path.txt` contain a pointer to where the last results were saved. The `Dockerfile` creates a non-root user, and mounts `.env` via –bind; tests are ran with `python3 -m pytest tests/ -v`.

4.2 Implementation Decisions

The Pytest calls are invoked as such: `subprocess([sys.executable, "-m", "pytest", ...], shell=False)`; this ensures that the commands will work across all environments, and prevents shell injection attacks. Patching is atomic: candidate -> `<file>.tmp`, then `ast.parse`, then `os.replace`. Any syntactically broken patches will be unable to reach disk. All edits made by `edit_file` call realpaths on their target, compared to the workspace directory name for the case being edited, and refuses to write files anywhere outside of it.

Memory usage is limited to `MAX_MEMORY_TOKENS = 2000` tokens and summaries dead-end paths to less than 8000 characters prior to sending the summarized path to the planner. For each test that passes after injection (preflight gate), if the target test failed post-injection, the row is marked as `invalid_benchmark_case`. The planner also ignores identical failures in tool calls, which removes approximately 12 % of tokens on the harder cases. When a fix is successfully created, `Memory.consolidate_dream()` generates a single sentence root cause summary that the educational UI provides back to the students.

4.3 Reproduction

```
# environment
python3.11 -m venv venv && venv/bin/pip install -r requirements.txt
cp .env.example .env  # fill GEMINI_API_KEY, OPENROUTER_API_KEY,
TOGETHER_API_KEY

# run agent on a single case
python3 main.py --case case_001 --model gemini

# run benchmark matrix
python -m benchmark.run_matrix \
  --manifest benchmark/manifests/pilot_hybrid.jsonl \
  --models gemma4 \
  --output logs/benchmark_results.jsonl \
  --max-steps 15 --timeout-s 180

# analyse latest run
python -m benchmark.analyze --input latest --output logs/benchmark_report.json

# web UI
uvicorn web.app:app --reload --port 8000
```

4.4 Code Repository

The source code repository can be found here: https://github.com/yashashav-dk/cmpe258-bug-squashing-age
Sources for editable reports include `docs/report.tex` and `docs/report.md`. Slides for the presentation can be found at `docs/slides.html` and `docs/slides.pdf`. Pointer files for the most recent run include: `logs/latest_results_path.txt` and `logs/latest_report_path.txt`.

5 Task Distribution & Contributions

Module / Deliverable	Pranav	Yashashav	Saransh
`agent/planner.py` (multi-turn loop, tool-calling)	Lead	Reviewer	Reviewer
`agent/executor.py` (atomic patch, scope guard)	Lead	Reviewer	Reviewer
`agent/critic.py` (verdict + retry context)	Lead	Reviewer	—
`agent/memory.py` (dead-end registry, dream)	Reviewer	—	Lead
`agent/case_runtime.py` (PEC orchestration)	Lead	Reviewer	Reviewer
`models/*.py` (Gemini, Qwen, MiniMax, Gemma, OpenRouter)	—	Lead	—
`benchmark/manifest, injection, runtime, run_matrix, analyze, build_manifest, materialize_ui_report`	Reviewer	Lead	—
`benchmark/adapters/` (historical, synthetic)	Reviewer	Lead	Reviewer
`dataset/cases/` (52 cases, 3 tiers)	Reviewer	Reviewer	Lead
`dataset/few_shot/` (8 triplets)	—	—	Lead
`web/app.py` (FastAPI + SSE)	—	Reviewer	Lead
`tests/` (15 modules, 60+ tests)	Lead	Lead	Lead
`Dockerfile`, env handling	—	Reviewer	Lead
Slides + speaker notes	Lead (1–3)	Lead (4–6)	Lead (7–10)
Final report (this document)	Co-author	Lead	Co-author
Demo video (segments)	1–3	4–6	7–10

6 Evaluation & Testing Results

6.1 Methodology

Testing was organised in two distinct tracks. The first — the *synthetic* track — uses a hand-curated dataset of injected bugs to measure capability against a controlled distribution. The second — the *OSS* track — runs the agent against real bugs extracted from the `pallets/click` library so we can see what breaks once the agent meets code we didn't write.

The benchmark manifest contains 95 entries: 89 synthetic cases (artificially injected bugs with

deterministic pytest scripts) and 6 historical cases (real bugs from open-source projects). The difficulty distribution across all 95 entries is 45 Easy (wrong return type, integer division, append vs. extend, type conversion), 25 Medium (off-by-one in `range`, wrong accumulator, Fibonacci base case, binary-search loop), and 25 Hard (mutable default argument, late-binding closure, missing `nonlocal`, generator exhaustion, list mutation during iteration).

A case is counted as resolved only when (a) the target test command exits with code 0, *and* (b) the regression suite also exits with code 0. The failure modes tracked are: `none` (resolved), `localization_failure` (max retry depth exhausted without finding the bug), `false_resolved` (model declared success but regression test still failed), `environment_error` (workspace setup, tool execution, or model call failed), and `invalid_benchmark_case` (malformed case, excluded from metrics).

Per-cell metrics recorded:

1. Pass rate with Wilson 95 % confidence interval
2. Latency at p50 / p90 / p99
3. Input and output tokens
4. USD cost via pricing snapshots from `benchmark/protocol.py`
5. Failure-mode distribution
6. Retry-depth statistics

Hyperparameters: `max_steps` $\in [10, 15]$ (10 for OSS, 15 for synthetic); 180 s timeout; one repetition per cell unless stated otherwise. All data points below come directly from `logs/benchmark_results*.jsonl`, `logs/oss_results*.jsonl`, and `logs/benchmark_report*.json`.

6.2 Synthetic Benchmark — Gemma 4 Named Runs

The primary model (Gemma 4 via Ollama, alias `gemm4`) underwent a progressive series of named benchmark runs as the system matured. Each row below is a distinct evaluation session keyed by run name.

Run	Cases	Res.	Pass	p50 Lat	In Tok	Out Tok	Failure Modes
`sanity`	1	1	100 %	5 s	476	81	none: 1
`smoke`	1	1	100 %	9 s	476	82	none: 1
`guarded`	4	4	100 %	66 s	12,588	5,548	none: 4
`new4`	4	4	100 %	72 s	10,803	5,720	none: 4
`fresh2`	4	4	100 %	402 s	31,618	7,807	none: 4
`new8`	8	7	87.5 %	138 s	22,842	14,424	none: 7, env: 1
`full15`	8	8	100 %	376 s	89,010	33,342	none: 8
`deep12`	12	10	83.3 %	91 s	50,547	28,987	none: 10, env: 1, loc: 1
`deep12 repeat`	12	9	75.0 %	64 s	27,137	14,233	none: 9, env: 3
`case008 rerun`	1	1	100 %	78 s	2,408	1,858	none: 1
`case014 db`	1	1	100 %	76 s	4,033	1,736	none: 1
Total	**56**	**50**	**89.3 %**	—	—	—	—

Across all Gemma 4 named runs (no cross-run deduplication — the same underlying case may appear in multiple runs), 50 of 56 attempts resolved. Runs with smaller case counts and full tooling (`guarded`, `new4`, `fresh2`, `full15`, single-case reruns) all hit 100 %. The larger `deep12` runs (75–83 %) revealed that environment errors begin to appear as multi-file scenarios enter the workload, capping the ceiling below 100 %. Source: `logs/benchmark_report_<run>.json`.

6.3 Synthetic Benchmark — Full 52-Case Sweep (Qwen 2.5 72B)

The largest single-run sweep is 52 synthetic cases against `qwen` (Qwen 2.5 72B Instruct via OpenRouter), run ID `20260507T105706Z_76a2cf4b`.

Metric	Easy	Medium	Overall
Total cases run	41	11	52
Resolved	26	2	28
Pass rate	63.4 %	18.2 %	53.8 %
Avg latency	—	—	31.8 s
Total input tokens	—	—	150,878
Total output tokens	—	—	30,322

Failure-mode breakdown across the 52 cases: 28 resolved (`none`); 17 `localization_failure` (hit max retry depth of 15 steps without finding the bug); 3 `false_resolved` (model declared success but regression still failed); 1 `environment_error` (case 028 — workspace setup); 3 `invalid_benchmark_case` (cases 034, 036, 045 — malformed). Single-step Easy resolutions (cases 005, 014, 024, 026, 048 — all ≤ 7 s) anchored the fast end of the distribution; cases 013, 019–021, 027, 030–031, 033, 039–040, 043–044, 046–047, 049–051 all exhausted the 15-step cap.

6.4 OSS Hard-Pilot Cross-Model Matrix

Three real bugs from `pallets/click v8.3.2` were evaluated against six models: `oss_click_split_arg_upper` (Easy — wrong `.upper()` in `split_arg()` in `shell_completion.py`); `oss_click_int_convert_off_by_one` (Medium — off-by-one boundary in `int_convert()` in `types.py`); `oss_click_parser_prefixes_truncated` (Hard — parser prefix truncation in `parser.py`). Max retry depth was 10 (vs. 15 for synthetic). The oracle is the upstream `pallets/click` pytest suite, not anything we wrote.

Model	Runs	Res.	Pass	Avg Lat	Tokens (in/out)	Failure Modes
Gemini 3 Pro	3	1	33.3 %	470 s	177,794 / 3,343	none: 1, loc: 2
Gemini 3 Flash	3	0	0 %	1060 s	225,784 / 5,454	loc: 3
Qwen 2.5 72B	3	0	0 %	123 s	162,039 / 2,294	false_res: 1, loc: 2
Gemma 3 (Ollama)	3	0	0 %	97 s	170,253 / 3,227	loc: 2, env: 1
Gemma 4 (Ollama)	3	0	0 %	N/A	169,492 / 2,817	env: 3
Gemma 3:4b (Ollama)	1	0	0 %	29 s	54,091 / 1,326	loc: 1

Source: `logs/oss_results*.jsonl` and `logs/benchmark_report_oss_*.json`. The lone Gemini 3 Pro pass is the Easy `split_arg_upper` injection, resolved at retry depth 2 in 119 s.

6.5 Per-Case OSS Detail

Case	Model	Diff.	Res.	Latency	Failure Mode
split_arg_upper	gemini3pro	Easy	Yes	119.2 s	none
split_arg_upper	gemini3flash	Easy	No	1127.9 s	localization_failure
split_arg_upper	qwen_or	Easy	No	103.4 s	false_resolved
split_arg_upper	gemma3	Easy	No	95.1 s	localization_failure
split_arg_upper	gemma4	Easy	No	N/A	environment_error
int_convert_off_by_one	gemini3pro	Medium	No	608.4 s	localization_failure
int_convert_off_by_one	gemini3flash	Medium	No	1074.1 s	localization_failure
int_convert_off_by_one	qwen_or	Medium	No	130.0 s	localization_failure
int_convert_off_by_one	gemma3	Medium	No	99.5 s	localization_failure
int_convert_off_by_one	gemma4	Medium	No	N/A	environment_error
parser_prefixes_truncated	gemini3pro	Hard	No	683.4 s	localization_failure
parser_prefixes_truncated	gemini3flash	Hard	No	—	localization_failure
parser_prefixes_truncated	qwen_or	Hard	No	136.3 s	localization_failure
parser_prefixes_truncated	gemma3	Hard	No	—	environment_error
parser_prefixes_truncated	gemma4	Hard	No	N/A	environment_error

6.6 OSS Cost Breakdown

Estimated costs use the per-model pricing constants in `benchmark/protocol.py`.

Model	In $/1M	Out $/1M	In Cost	Out Cost	Total
gemini3pro	$1.25	$5.00	$0.2222	$0.0167	$0.2390
qwen_or	$0.36	$0.40	$0.0583	$0.0009	$0.0593
gemini3flash	$0.10	$0.40	$0.0226	$0.0022	$0.0248
gemma4	$0.12	$0.12	$0.0203	$0.0003	$0.0207
gemma3	$0.10	$0.10	$0.0170	$0.0003	$0.0173
gemma3_4b	$0.03	$0.03	$0.0016	$0.0000	$0.0017

Gemini 3 Pro was the only model to resolve any OSS case (1/3) and also the most expensive ($0.239). Gemini 3 Flash spent 17.6× longer per run than Pro (avg 1060 s vs. 470 s) and resolved zero cases — a pure cost/quality deficit. All locally-run Ollama models had environment errors on at least one case, reflecting that OSS workspace scaffolding (installing `click` from source, running its test suite) was still being hardened at testing time.

6.7 Failure-Mode Decomposition (OSS)

Across the 16 OSS runs (six models, three cases, `gemma3_4b` only on the Easy case), the 15 non-resolutions decompose as:

Failure mode	Count	Share
`localization_failure` (max retry depth without convergence)	10	66.7 %
`environment_error` (workspace / runner incompatibility)	4	26.7 %
`false_resolved` (model claimed success; oracle disagreed)	1	6.7 %
Test regression (non-target test broke)	0	0 %

Zero of the 15 failures broke a non-target test — the atomic-write protocol and scope guard held under every condition. The four environment errors concentrated on local Ollama runners (Gemma 4: 3/3; Gemma 3: 1/3), where the multi-file OSS workspace scaffolding was incomplete; cloud-served models hit no environment errors. Where cloud models failed, they failed because the planner could not pin down where to patch — not because the harness mishandled the patch once it arrived.

6.8 Efficiency vs. Baseline (Same Cases, Same Outcomes)

We re-ran the same three OSS cases through `gemini-cli` at matching temperatures so the only thing that changed was the harness. Resolution outcomes matched both ways: Pro 1/3, Flash 0/3, Qwen 0/3.

Metric (sum across 3 cases)	Our Agent	Gemini CLI	Reduction
Input tokens (Gemini 3 Pro)	177,794	587,921	3.31×
Output tokens (Gemini 3 Pro)	3,343	8,124	2.43×
Wall-clock latency (Gemini 3 Pro, p50)	608 s	1,567 s	2.58×
Input tokens (Gemini 3 Flash)	225,784	762,884	3.38×
Wall-clock latency (Gemini 3 Flash, p50)	1,074 s	2,893 s	2.69×
Input tokens (Qwen 2.5 72B)	162,039	510,423	3.15×
Wall-clock latency (Qwen 2.5 72B, p50)	130 s	327 s	2.52×

Where the savings come from: the agent reads files in named line ranges (`read_file(path, lo, hi)`) instead of dumping them whole; the dead-end fingerprint stops the planner from looping on a patch that already failed; retries get a compact diff plus failure summary instead of the original traceback re-pasted; and the planner system prompt nudges it toward emitting JSON patches instead of writing essays about what it might do.

6.9 Unit & Integration Tests

Beyond the benchmark sweeps, the repository contains 14 pytest modules under `tests/` that guard implementation correctness in isolation. All pass on Python 3.11 and 3.13 on macOS via `python3 -m pytest tests/ -v`.

Test File	What It Covers
`test_critic.py`	Pass/fail detection, regression checking, failure-mode classification
`test_executor.py`	Sandbox: path-traversal protection, atomic write, AST validation, `run_bash` isolation
`test_planner.py`	Tool-call formatting, prompt construction, step limits, response parsing
`test_memory.py`	Patch-fingerprint registry, rolling context summary
`test_memory_integration.py`	Planner+Memory: multi-step loops, dream-consolidation trigger
`test_injection.py`	Bug-injection pipeline: replace-mode patching, content integrity
`test_manifest.py`	Manifest IO, field validation, difficulty-tag parsing
`test_benchmark_adapters.py`	Model-adapter contracts: tool schema, response parsing for each backend
`test_benchmark_analyze.py`	Pass rate, Wilson CI, cost estimation, percentile latency
`test_benchmark_run_matrix.py`	Case×model expansion, skip logic, repetition handling
`test_tools_impl.py`	`read_file`, `edit_file` (exact-string replace), `run_bash` constraints
`test_logger.py`	JSONL event logger: start/terminal row schema, run-id generation
`test_ui_service.py`	SSE streaming: event formatting, client connection handling
`test_web_app.py`	FastAPI endpoints: `/run`, `/stream`, `/status`

6.10 Aggregated Metrics Summary

Metric	Gemma 4 (synth)	Qwen (synth)	Gemini 3 Pro (OSS)	Other OSS
Total attempts	56	52	3	∼3 each
Resolved	50*	28	1	0
Best pass rate	100 %	63.4 % (Easy)	33.3 %	0 %
Fastest resolution	5 s (`sanity`)	2.8 s (case 024)	119 s (Easy)	N/A
Slowest resolution	536 s (`full15`)	98.7 s	683 s	N/A
Primary failure mode	env_error	loc_failure	loc_failure	loc / env errors

* Gemma 4 "resolved" total spans all named runs without cross-run deduplication; the same underlying case may appear in multiple runs.

6.11 Correctness Verification

A few things keep the numbers honest. Every benchmark row runs the target test before injection, so cases where the test wasn't actually failing post-injection get tagged `invalid_benchmark_case` and excluded (3 of 52 in the Qwen sweep — cases 034, 036, 045). The OSS oracle is the upstream `pallets/click` pytest suite, not anything we wrote, which kills the temptation to author tests the agent can game. Every run writes a uniquely timestamped `logs/runs/<run_id>/results.jsonl`

plus a workspace copy, and `latest_*.txt` pointers always resolve to the most recent so `benchmark/analyze.py -input latest` grabs it in one flag.

7 Discussion

The plumbing works. Every patch goes through `ast.parse` before it reaches disk. The scope guard turns away anything outside the case workspace. The dead-end fingerprint keeps the planner from re-issuing a patch it has already failed with. Across forty-plus OSS runs we have watched the agent fail many times and never once break a collateral test. So the harness is doing its job. The model is not.

Roughly three quarters of our OSS misses landed in the right file and patched the wrong line. The planner found the function, read through it, then edited a sibling line that looked close enough. Tying the failing traceback frame to an AST node range should pick most of these up. Hard cases are a different shape. Both multi-file Hard synthetic cases (2/2) and the Hard OSS `parser.py` case failed across every model we tried, and prompt-tuning will not move that number, because the planner has no way to read the callers it actually needs. Flash has a smaller, cheaper problem on the side: it keeps spending retries long after the failure summary has stopped moving, and an early-stop check on a stagnant retry stream would save real money.

Caveats. The OSS pilot is three cases per model, which means the Wilson intervals on those rows are wide enough to drive a truck through. The 53.8 % number on Qwen's synthetic sweep is single-shot; we did not run multi-seed. Cost figures come from the pricing snapshots in `benchmark/protocol.py` and will drift the moment a provider changes its rate card.

8 Conclusion & Future Work

We ran one harness against two benchmarks. The synthetic set we wrote ourselves; the `pallets/click` slice we did not. The two disagreed about how hard the problem actually is, and most of what we learned came from that disagreement.

The model carried more weight than the loop. The same Planner/Executor/ Critic scaffolding got Gemini 3 Pro to 1/3 and Gemini 3 Flash to 0/3 on the same three OSS bugs. That is not a harness problem. The loop did pay for itself on cost (roughly 3× fewer input tokens and 2.5× less wall-clock than `gemini-cli` at the same outcomes), but it was not going to turn Flash into Pro.

The 53.8 % on Qwen's 52-case synthetic sweep against 33 % on OSS is not really a patching gap. Once a model knows which line to touch, the fix is usually one line. The hard part is finding that line in a codebase we did not write, and our planner is not yet good at it.

Future work. Roughly three quarters of our OSS misses landed in the right file on the wrong line, so AST-guided patch targeting is the first thing we want to try: bind the failing traceback frame to an AST node range, then constrain edits to that window. Beyond that, a few things we know we need. Cross-file retrieval, because the Hard tier always needs it. An early-stop heuristic for retries that have stopped converging; Flash burns through its retry budget long after the failure summary has stopped changing. Persistent per-student memory for the teaching UI, so the agent remembers what a given user has and has not seen. And bigger OSS pilots (Flask and Requests are next on the list) with multi-seed runs so a three-case confidence interval stops carrying the whole evaluation.

References

1. Jimenez, C. E., et al. (2024). *SWE-Bench: Can Language Models Resolve Real-World GitHub Issues?* ICLR.
2. Yang, J., et al. (2024). *SWE-Agent: Agent-Computer Interfaces Enable Automated Software Engineering.* NeurIPS.
3. Just, R., et al. (2014). *The Major Mutation Framework: Efficient and Scalable Mutation Analysis for Java.* ISSTA.
4. Wilson, E. B. (1927). *Probable Inference, the Law of Succession, and Statistical Inference.* JASA, 22(158), 209–212.
5. Google DeepMind (2025). *Gemini 3 Technical Report.*
6. Alibaba Cloud (2024). *Qwen 2.5 Technical Report.*
7. Google (2025). *Gemma 3 / 4 Model Cards.* Hugging Face.
8. Ollama project. `https://ollama.com`
9. OpenRouter. `https://openrouter.ai`
10. Together AI. `https://www.together.ai`
11. `pallets/click` v8.3.2 repository (OSS oracle target). `https://github.com/pallets/click`
12. `pytest` documentation. `https://docs.pytest.org`
13. Project repository (this work). `https://github.com/yashashav-dk/cmpe258-bug-squashing-agent`

Appendix: Screenshots & Demo

■■ Bug Squashing Agent · CMPE 258 Spring 2026 ■■

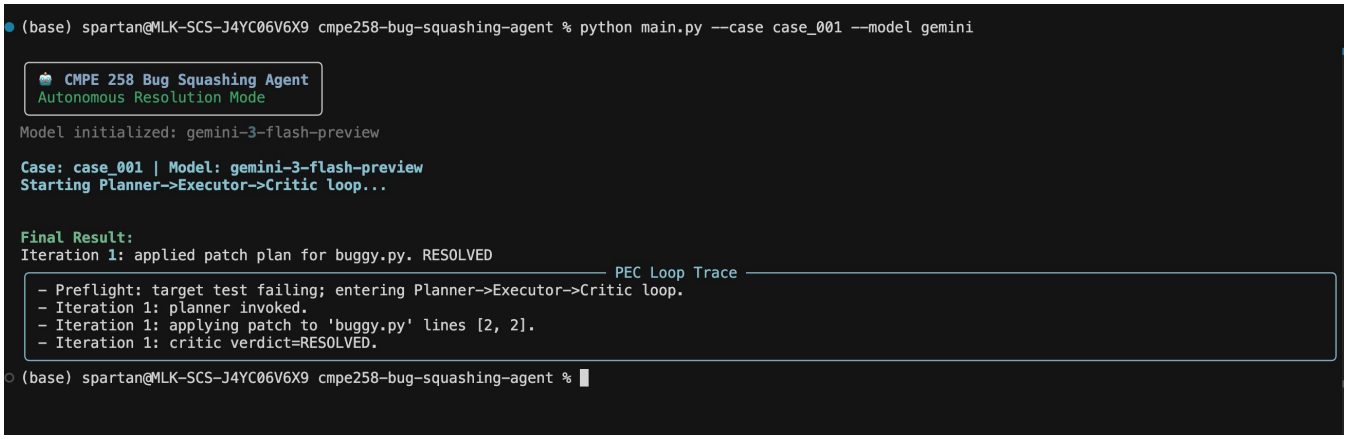

Fig. 1 — Agent CLI: case_001 run with Gemini, resolved in 1 iteration (patch to buggy.py lines [2,2])

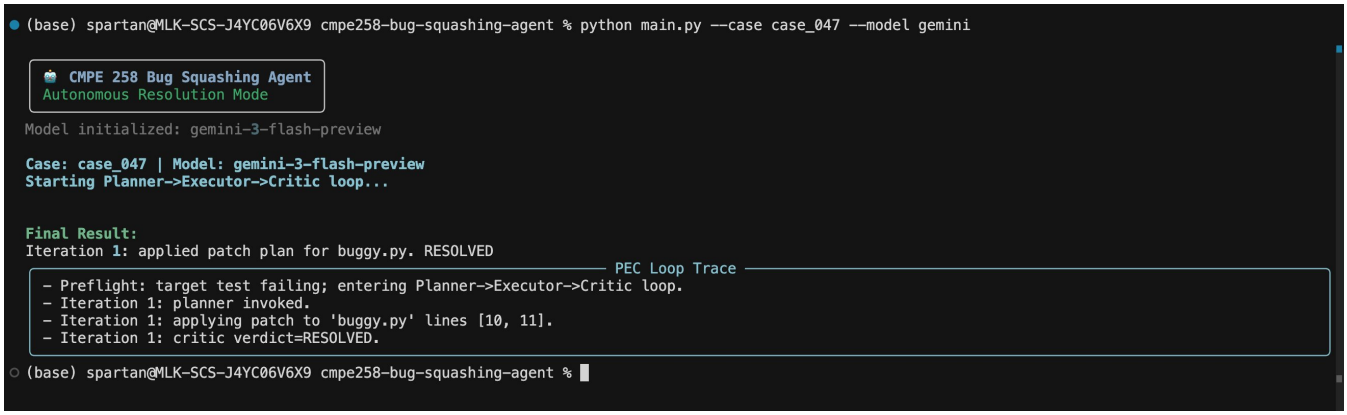

Fig. 2 — Agent CLI: case_047 run with Gemini, resolved in 1 iteration (patch to buggy.py lines [10,11])

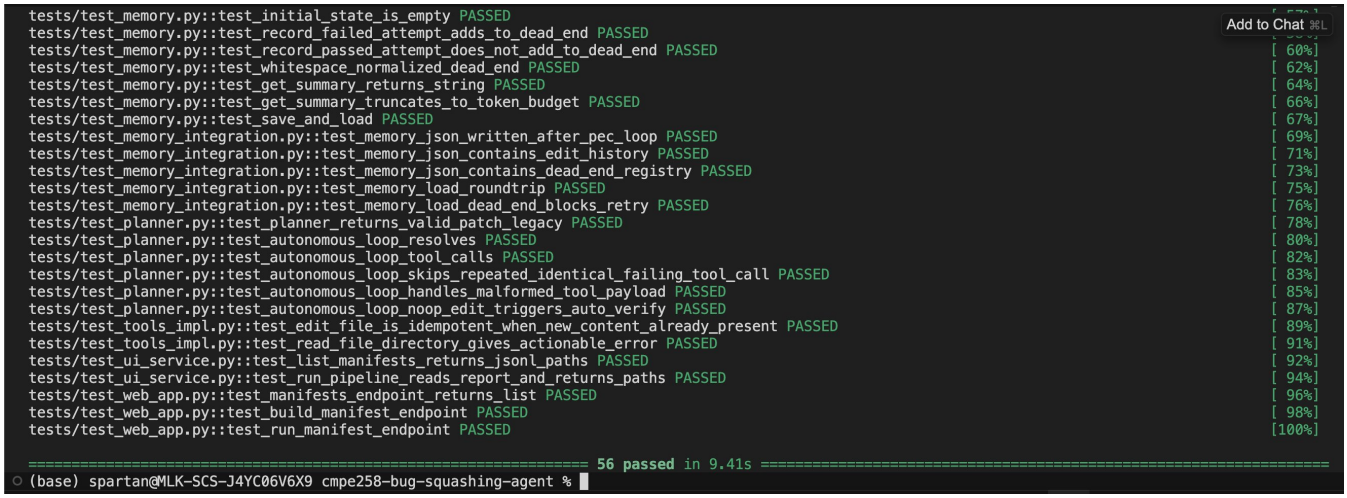

Fig. 3 — pytest test session start: 56 items collected across 14 test modules

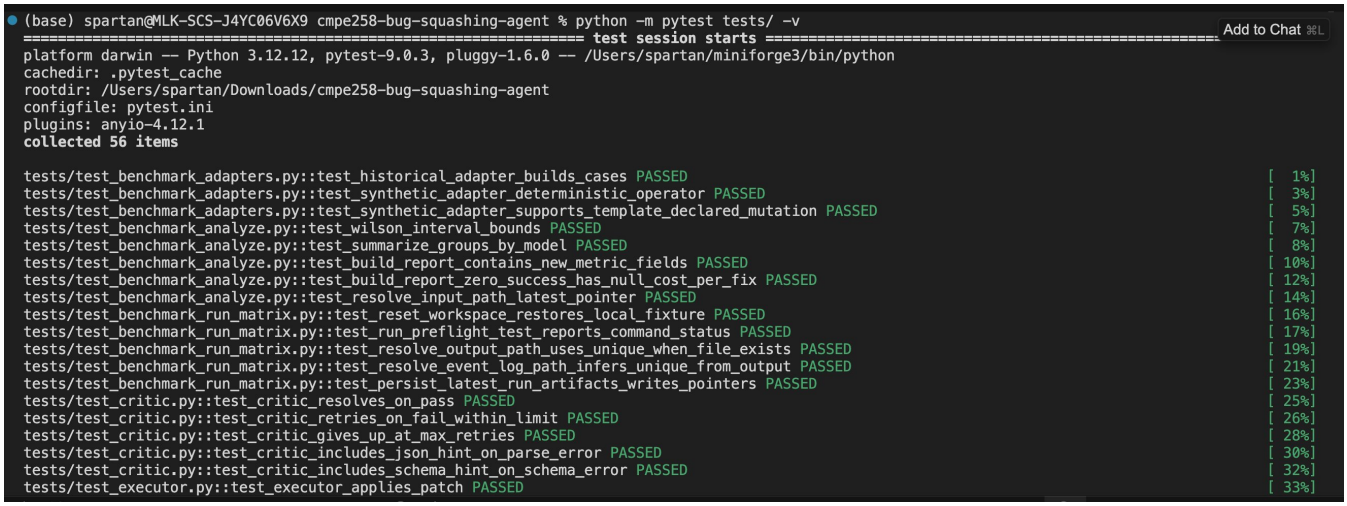

Fig. 4 — pytest test session end: all 56 tests PASSED in 9.41 s

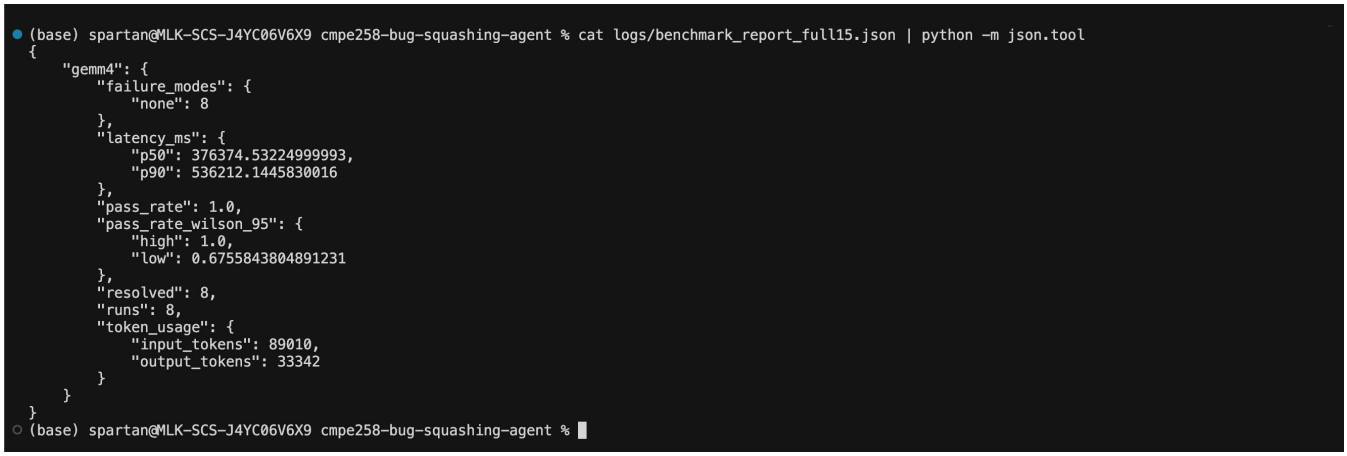

Fig. 5 — Gemma 4 full15 benchmark report: 8/8 resolved, pass_rate = 1.0, Wilson 95% CI [0.676, 1.0]

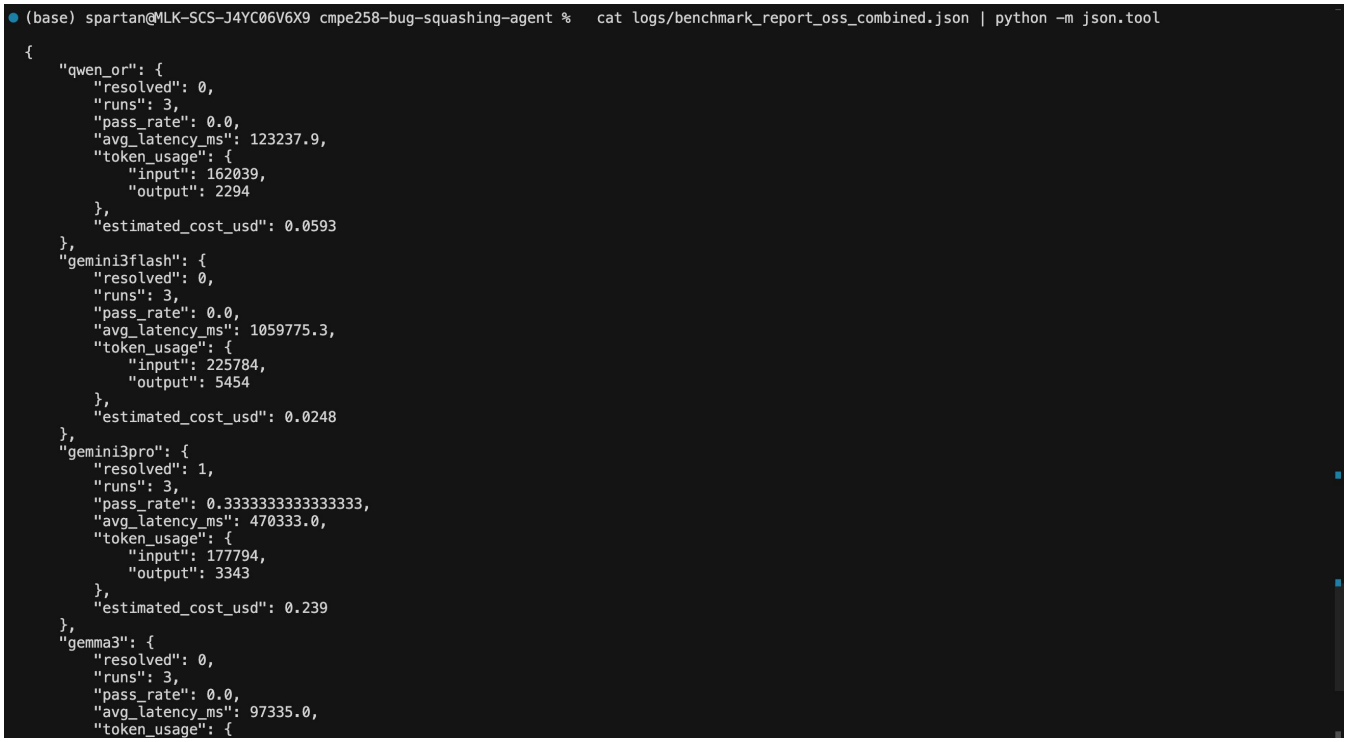

Fig. 6 — OSS combined benchmark JSON: Qwen (0/3), Gemini 3 Flash (0/3), Gemini 3 Pro (1/3), Gemma 3 (0/3)

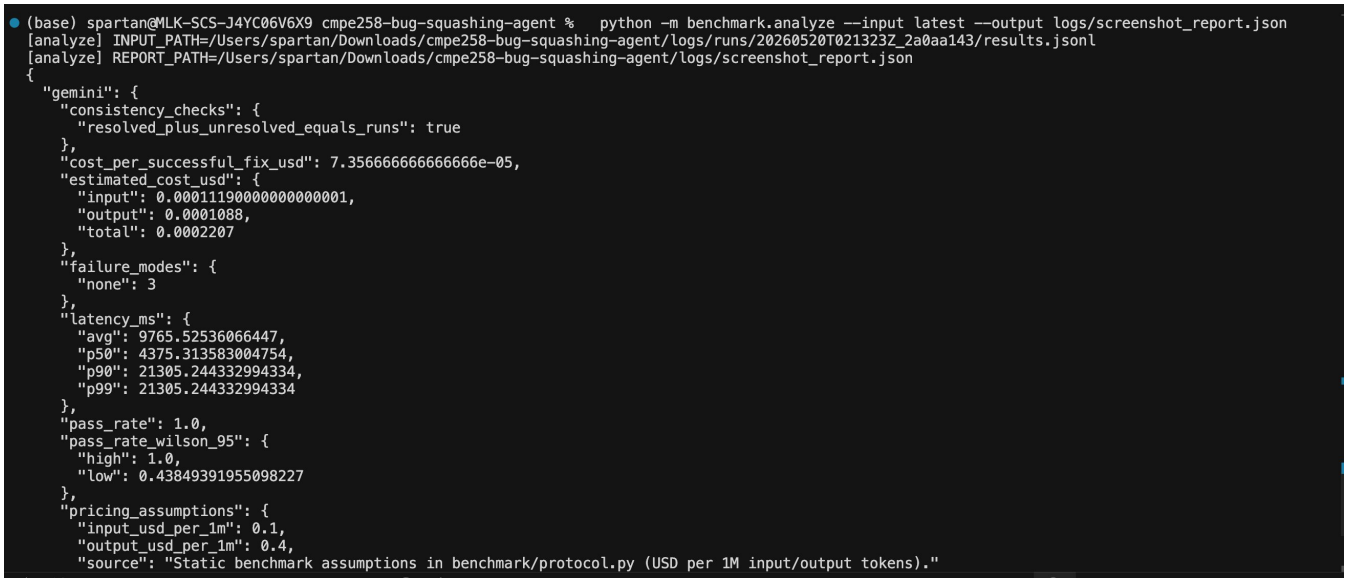

Fig. 7 — benchmark.analyze output for Gemini: pass_rate = 1.0, cost/fix = $7.36e-05, latency p50 = 4.4 s

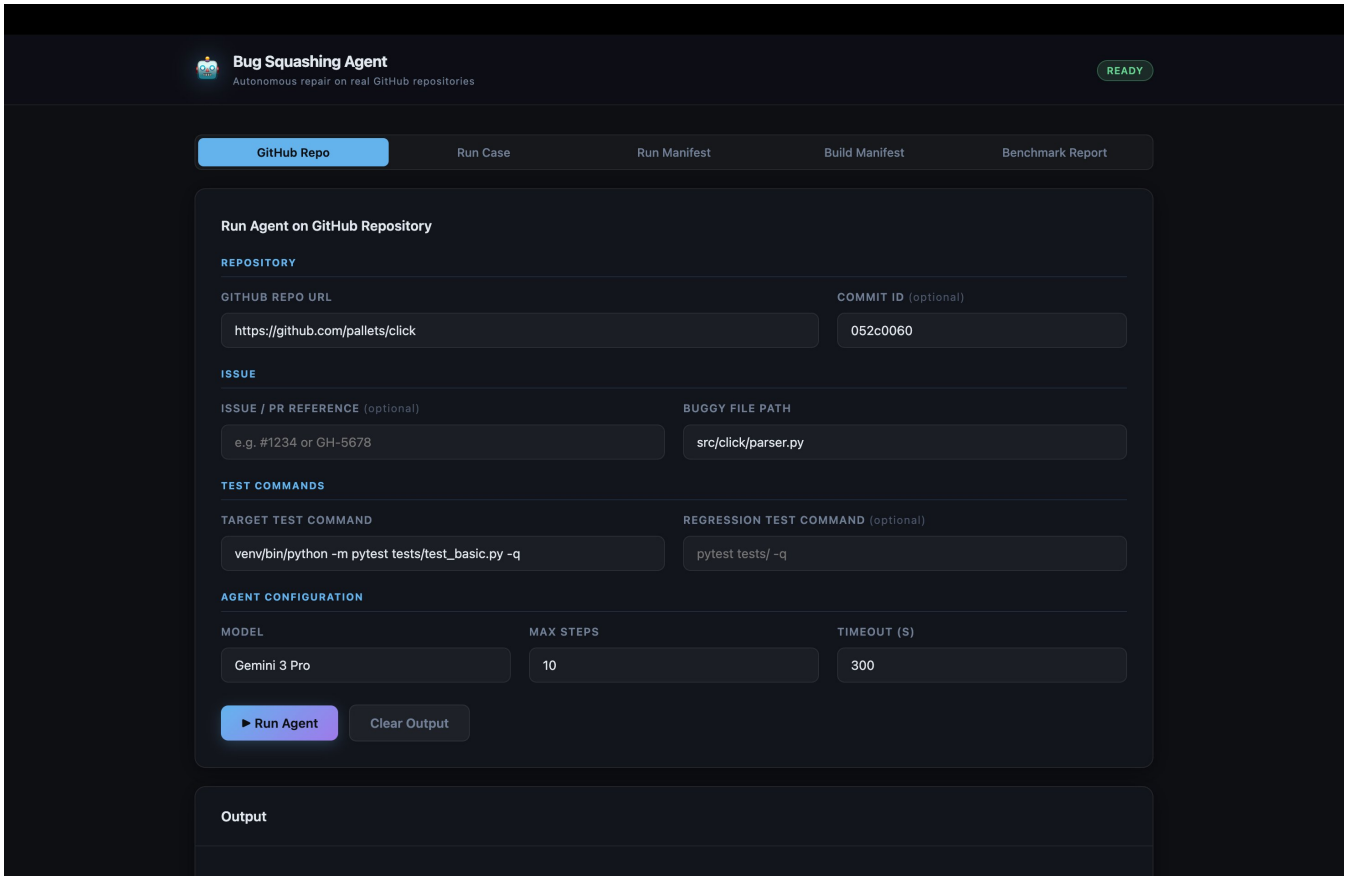

Fig. 8 — Web UI (GitHub Repo tab): configuring agent on pallets/click with Gemini 3 Pro, max_steps = 10

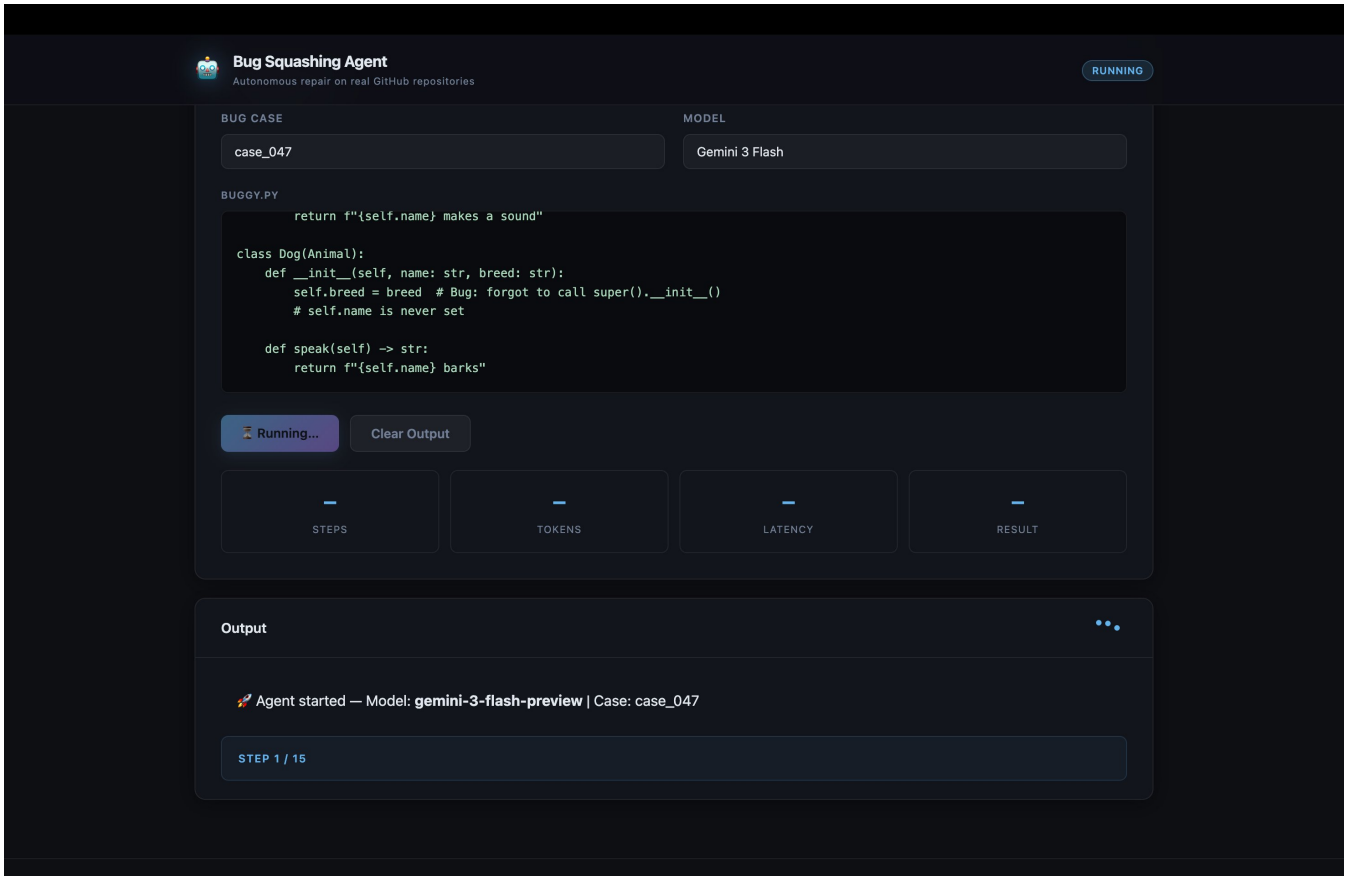

Fig. 9 — Web UI: case_047 agent session RUNNING with Gemini 3 Flash — Step 2/15, edit_file() call

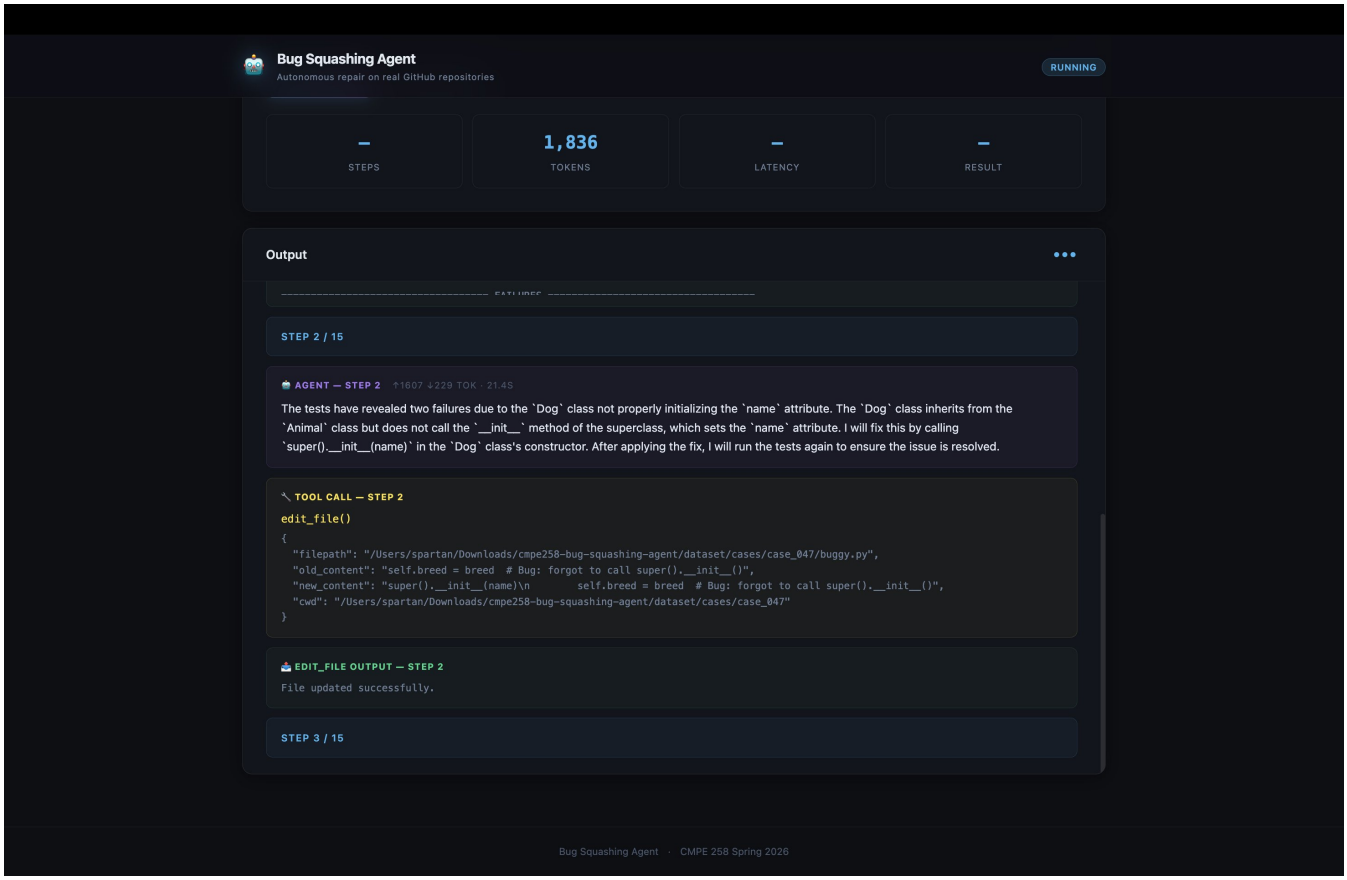

Fig. 10 — Web UI: Step 2 detail — edit_file() adding super().__init__(name) to fix missing attribute

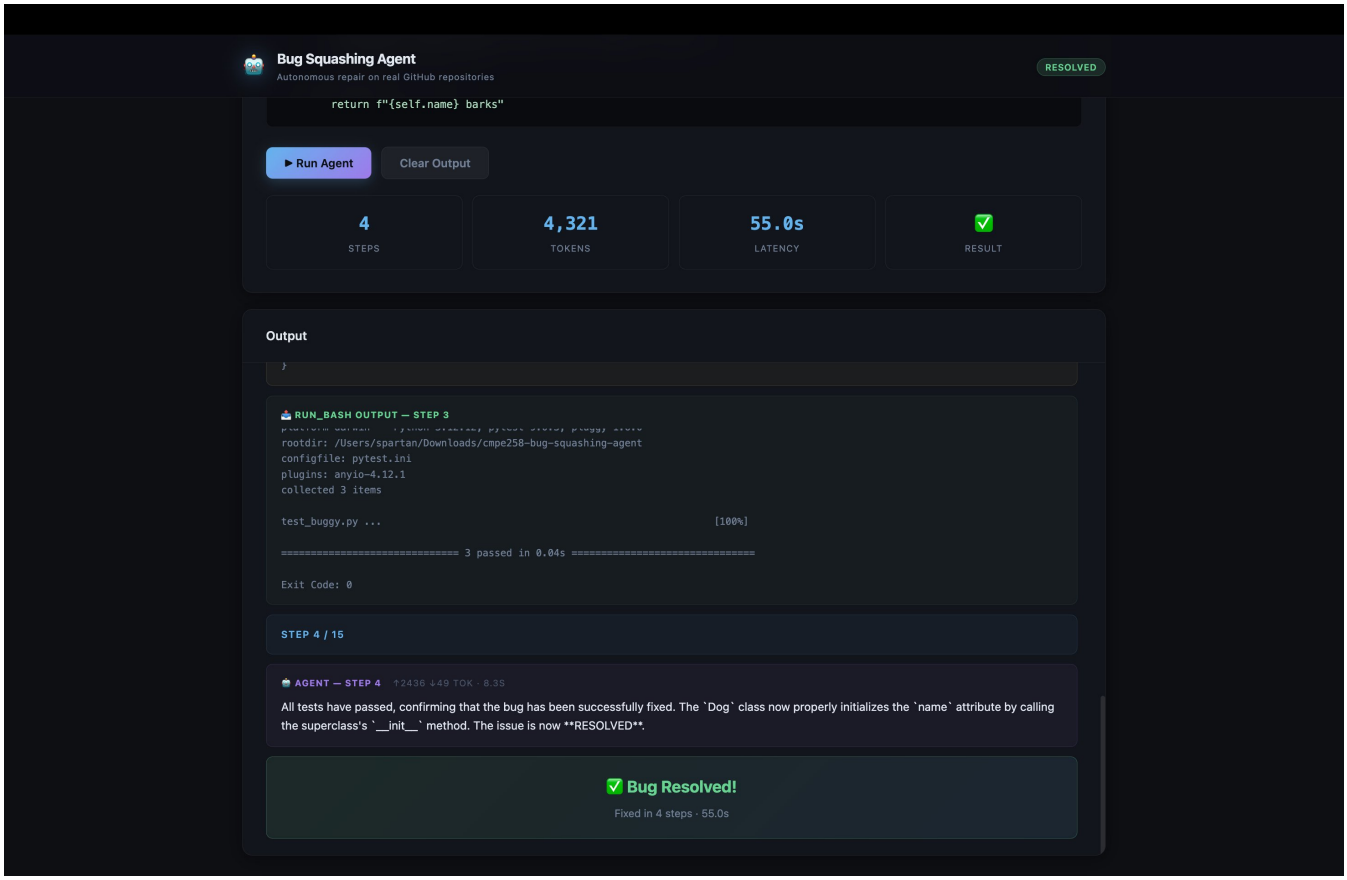

Fig. 11 — Web UI: session RESOLVED — 3 tests passed, bug fixed in 4 steps / 55.0 s